

Microservices overview and communication flow

The platform's backend is divided into five core microservices, each encapsulating a specific domain and operating independently. This modularity ensures better scalability, team ownership, and fault isolation. Figure 2 illustrates the code snippets for the microservices, along with their respective databases and responsibilities. The complete source code for this project is available in our GitHub repository at <https://github.com/pink-27/ThriftStore-For-College-Students>. Interested readers are encouraged to clone the repository and explore the application.

▪ Core microservices

Product service: Handles product listings, categories, and seller-specific data. **Database:** MongoDB

Order service: Manages order placement, payment status, and tracking. **Database:** PostgreSQL

Wishlist service: Allows users to save and manage products of interest. **Database:** MongoDB

Chat service: Enables real-time buyer-seller communication. **Tech:** WebSocket + MongoDB + Kafka

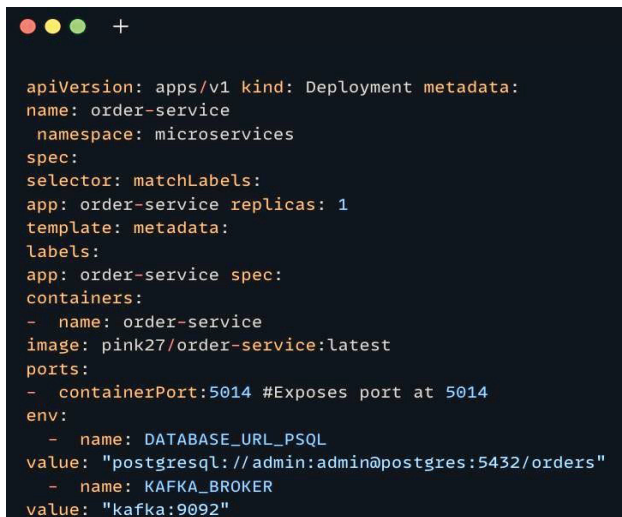
Notification service: Sends system-generated alerts triggered by Kafka events. **Tech:** Kafka + WebSocket (for frontend push)

▪ Inter-service communication

A hybrid communication model is implemented for optimal responsiveness and scalability.

Synchronous (REST APIs): Used for direct data access and CRUD operations.

Example: Wishlist calls Product service to fetch product details.



```
apiVersion: apps/v1 kind: Deployment metadata:
  name: order-service
  namespace: microservices
spec:
  selector: matchLabels:
    app: order-service replicas: 1
  template: metadata:
    labels:
      app: order-service spec:
    containers:
      - name: order-service
        image: pink27/order-service:latest
        ports:
          - containerPort:5014 #Exposes port at 5014
        env:
          - name: DATABASE_URL_PSQL
            value: "postgresql://admin:admin@postgres:5432/orders"
          - name: KAFKA_BROKER
            value: "kafka:9092"
```

Figure 2: Sample Kubernetes Deployment Manifest (illustrative only; sensitive values should be externalized via environment variables or Kubernetes Secrets).

Asynchronous (Apache Kafka): Enables decoupled, real-time workflows.

Example: When an order is placed, an order-event is published to Kafka. Notifications and Chat services consume this event.

Each service subscribes only to the topics relevant to its business logic (e.g., order-events, chat-events, notification-events), ensuring clean separation and scalability. Figure 2 depicts the sequence of operations in an e-commerce microservices architecture for order management. It shows how the OrderService interacts synchronously via REST with the ProductService and asynchronously via Kafka during viewing, creating, and cancelling orders. The flow involves retrieving product details, creating orders, publishing order events, and updating order status, all involving coordinated communication between users, services, and databases.

This setup not only enhances performance under load but also simplifies debugging and horizontal scaling. REST handles immediate tasks, while Kafka ensures reactive behaviour and eventual consistency.

Deployment architecture

The entire application is containerised using Docker and orchestrated on a local Kubernetes cluster via Minikube. This setup closely mirrors production environments, offering service resilience, scalability, and ease of management.

Each microservice runs in its own Docker container, exposing only the necessary ports and packaged with minimal dependencies. These containers are managed via Kubernetes Deployments, which ensure desired pod states and handle restart policies, scaling, and updates.

Key Kubernetes components are:

Pods and Deployments: Each service has its own deployment definition and can be scaled independently.

Example: The Product service runs with 2 to 4 replicas based on CPU load.

ClusterIP services: Internal DNS based service discovery, allowing one microservice to reach another by name; e.g., `http://order-service:5020/api/orders`.

ConfigMaps and Secrets: Used to inject environment-specific variables like DB URIs, Clerk keys, Kafka broker addresses, etc, into each container securely.

Horizontal Pod Autoscaler (HPA): Automatically adjusts the number of running pods based on resource usage metrics. For example, Product service scales based on CPU utilisation under high traffic.

Figure 4 shows the Kubernetes deployment of a microservices based architecture within a Minikube cluster. It includes services like Product, Wishlist, Chat, Notification, and Order, all communicating through Kafka as the message broker. PostgreSQL handles persistent storage, while